A generalized framework for heterogeneous computing

Dual Degree Project
Stage 2

by

Aditya Bhosale
140010009

under the guidance of

Prof. Prabhu Ramachandran

Department of Aerospace Engineering
Indian Institute of Technology, Bombay
Mumbai 400 076

Abstract

In recent years, GPUs, which were first introduced to typically handle computations only for computer graphics, are being increasingly used for targeting more general-purpose computing applications. These days systems with 8-12 GPUs on a single node have become quite common. This combined with the powerful CPUs available in the market today makes it important to develop frameworks that can harness the power of these systems in the most optimal way. In this report we present one such framework that can be used for running parallel applications on hybrid systems. We present our load balancing strategy which supports weighted objects and processes. We also briefly explore Compyle which is heavily used in our implementation and is also required to be used by the users for writing applications using our framework.

Contents

1 Introduction **1**

2 PyZoltan **3**

2.1 Dynamic load balancing 3

2.2 Unstructured point-to-point communication 3

2.3 Distributed data directories 4

2.4 CUDA aware MPI 5

3 Load balancing **6**

3.1 Load Balancer 6

3.1.1 Recursive coordinate bisection 6

3.2 Cell based load balancing 7

4 The framework **10**

4.1 Communication framework 10

4.2 Cell Manager 10

4.3 Partition Manager 11

4.4 Cell Map 11

4.5 Object Exchange 11

5 Analysis **14**

5.1 Load balancing 14

5.2 Migration of objects 14

6 Future work **18**

A Compyle **19**

Chapter 1

Introduction

Particle methods are increasingly used to simulate a variety of problems, particulary problems with complex boundary conditions or problems with large boundary displacements. A variety of particle methods have been developed over the years including MLS [Savitzky and Golay, 1964], SPH [Gingold and Monaghan, 1977], DSMC [Bird, 1976], etc. Particle methods generally work by discretizing the flow properties into particles. The particle properties are then evolved depending upon their governing equations. These particles interact with their neighboring particles. For example, in SPH, the particle properties at a particle location is given by the weighted average of it's neighboring particles. Thus, the neighbors of each particle need to be found at each time step. The neighbor finding part is one of the major bottlenecks of particle method based simulations. For example, in SPH, around 40% of the total simulation time is spent on finding neighbors. We previously have looked at efficient nearest neighbor finding algorithms in serial [Bhosale, 2016].

In recent years, GPUs, which were first introduced to typically handle computations only for computer graphics, are being increasingly used for targeting more general-purpose computing applications. Figure 1.1 shows the floating point operations per second of GPUs vs CPUs over the years. It can be seen that the GPU has evolved into a highly parallel processor with tremendous computational horsepower.

GPUs have been widely used for SPH simulations in recent years [Xia and Liang, 2016] [Goswami et al., 2010] [Krog and Elster, 2012]. We have previously also looked at various algorithms for efficient nearest neighbor searching on the GPU [Bhosale, 2017]. We developed a uniform grid based algorithm using space filling curves to efficiently find neighbors for constant smoothing lengths. We also introduced an algorithm based on a heirarchy of grids to efficiently find nearest neighbors on variable smoothing lengths that was nearly twice as fast as the uniform grid based algorithm.

While GPUs provide a platform to speed up computations to a large degree, we fail to utilize every last bit of compute power from the server or PC or cluster on which our simulation runs if we only use the GPUs for computation while the CPU cores remain idle. Heterogeneous computing is a paradigm that aims to utilize both CPU/GPU architectures and achieve load balancing to minimize idle time on both platforms. Heterogeneous computing, however presents new challenges in form of minimizing data transfer between different PUs and different nodes

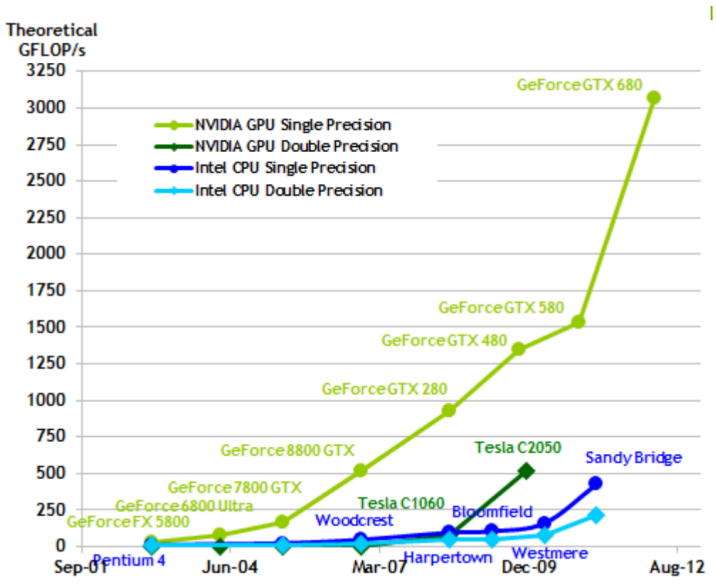


Figure 1.1: Comparison of GFLOP/s for GPUs and CPUs over the years. Image from [NVIDIA, 2013]

while making sure that each PU has the appropriate load according to it's computational ability. This is a non-trivial problem to solve as CPUs and GPUs have a large difference in compute capabilities and the communication overhead plays a major role in overall performance of the algorithm.

We looked at the load balancing strategy of DualSPHysics [Domínguez et al., 2013] and found that while their strategy is convenient and easy to implement, it is not the optimal load balancing strategy. We found that a geometric partitioner would be more efficient for load balancing. We will look at PyZoltan in chapter 2. While PyZoltan is efficient and easy to use, the library does not support GPUs. Thus, we have tried to implement a simpler version of Zoltan in pure python using mpi4py [Dalcin et al., 2011] with support for both CPUs and GPUs. We will also look at CUDA aware MPI which we use for data transfers in our framework. We will then look at the load balancing algorithm we use in chapter 3. Then we will look at the design of our framework in chapter 4. In appendix A we look at Compyle, a framework for executing pure python on a variety of platforms. We use Compyle to write kernels that execute on CPUs as well as GPUs.

Our aim in this project was to design a framework to provide support for heterogeneous computing in PySPH. While the support to PySPH is still incomplete, we have been able to develop a framework that can allow users to write a variety of heterogeneous applications in it's current state.

Chapter 2

PyZoltan

The Zoltan library is a collection of data management services for parallel, unstructured, adaptive and dynamic applications. Out of the many features that Zoltan provides, the PyZoltan library [Puri et al., 2013] wraps the specific methods to support the certain features in PySPH [Prabhu Ramachandran, 2016] that we will see in the following sections.

2.1 Dynamic load balancing

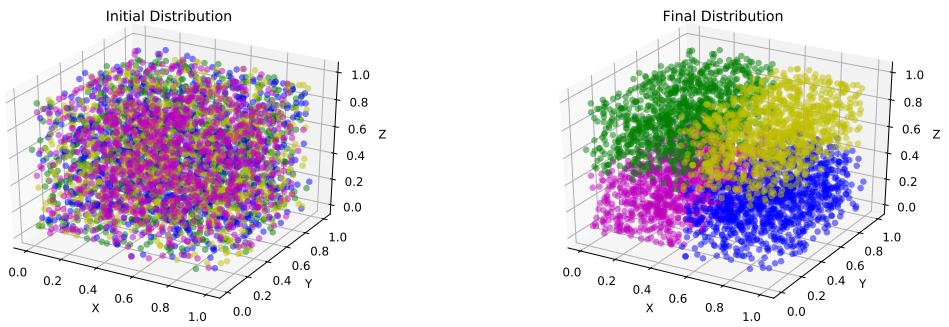
Zoltan provides a variety of geometric load balancing algorithms. Figure 2.1 shows an example of result of load balancing with 4 processors for a random initial distribution of particles using the RCB (Recursive Coordinate Bisection) method. The load balance method sets a number of arrays indicating the objects that need to be imported and exported.

Situations commonly occur where we only have a list of objects that need to be exported and we do not have a list of objects to be imported. This is a complicated situation as buffers of appropriate size need to be allocated on the receiving end before actually sending the data. Zoltan provides an API to invert the export list to get the import list. Meaning, when the list of objects to be exported from a process is known, Zoltan will find the list of objects to be imported to that process from remote processes. This also works the other way when you have a list of objects to be imported but not the list of objects to be exported, however such a situation is rare.

Zoltan is used for dynamic load balancing in PySPH. The particles are first binned into cells of size kh, where k is the radius scale and h is the smoothing length. The cells are then used for load balancing. The number of particles in each cell is stored. The cells are weighted according to the number of particles in each cell.

2.2 Unstructured point-to-point communication

Once we have the import and export lists, the next step is to structure the communication to exchange the data using MPI. Zoltan provides an API for easier exchange of data between processors via the `ZComm` class. The `ZComm` object is instantiated using the number of objects to be sent and the list of processes to which the objects are to be sent. The number of objects to be



(a) Initial assignment of particles to processors before load balancing

(b) Final assignment of particles to processors after load balancing

Figure 2.1

received by a process can be found using the `ZComm.nreturn` attribute and the buffer of corresponding size can be allocated. The transfer can then be carried out using the `ZComm.Comm_Do` method.

A number of times, objects are shared with remote processes and the value of these objects are updated in the remote process. These objects are now supposed to be communicated back to their source. Zoltan provides an API for doing this through the method `ZComm.Comm_Do_Reverse`.

Thus, using Zoltan, users don't have to make explicit low-level MPI send and receive calls. All that the users have to do is correctly allocate the data on the receiving side.

2.3 Distributed data directories

The Zoltan Distributed data directory allows users to conveniently locate data on remote processes. After the instantiation of the distributed directory, each process updates the directory with the global IDs and the data in that process. Now when a global ID is queried from any process, zoltan finds the process to which the data belongs and sends the data to the process from which it was queried.

Zoltan provides a clean API for the above mentioned functions and more. However, zoltan does not have any GPU support. Also, the CUDA aware MPI that we will look at in the next section cannot automatically provide GPU support as the computations in Zoltan are all done serially and written in pure C. Thus, we found that it was unavoidable to write our own simpler and light weight implementation of Zoltan. It is also extremely useful as now we have a pure Python, simplistic version of Zoltan with minimal dependencies apart from the fact that we also now support multiple GPUs, multiple CPUs as well as CPU-GPU hybrid configurations.

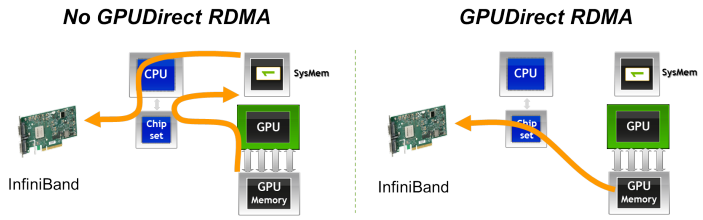


Figure 2.2: Illustration of internodal GPU-GPU transfers using GPU Direct. Image from [NVIDIA, 2013]

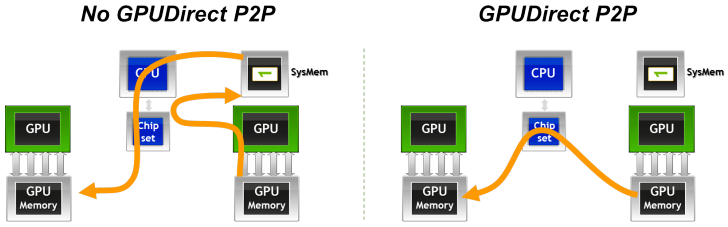


Figure 2.3: Illustration of peer-to-peer GPU-GPU transfers using GPU Direct. Image from [NVIDIA, 2013]

2.4 CUDA aware MPI

One of the bottlenecks of distributed applications is the data transfers. This problem is further worsened when using GPUs as the compute time is reduced while the data transfer times are increased as an additional layer of data transfer from the GPU to the CPU is now added.

Most MPI implementations have support for a CUDA aware MPI. This allows users to write data transfer calls for GPU memory identical to that of CPU data transfer calls. That is, given the source and destination addresses on the GPUs, MPI does the appropriate transfer of memory. The same calls can also be used for memory transfer between GPUs and CPUs. The CUDA aware MPI also makes use of technologies like GPU Direct for efficient transfer of data between GPUs on the same node as well as those on different nodes. Thus, using CUDA aware MPI we can eliminate the intermediate data transfer from GPU to CPU for GPU-GPU transfers.

For inter-node communications with RDMA support, the buffers are directly sent to the network adapter without staging through host memory as can be seen in figure 2.2. For peer-to-peer transfers, buffers can be copied directly from one GPU memory to other without passing through the host memory as can be seen in figure 2.3.

We use MPI for all data transfers in our framework. We rely on users having a CUDA enabled MPI installation to be able to use multiple GPUs. If a CUDA enabled MPI installation is unavailable, the same framework can still be used for multiple CPUs.

Chapter 3

Load balancing

The major challenge in implementing support for heterogeneous platforms is to efficiently distribute the load across PUs while minimizing the transfer of data between them. DualSPHysics [Domínguez et al., 2013], a popular SPH implementation partitions the domain perpendicular to the longest dimension. We had found out in stage 1 that while this is convenient, it's not the most efficient way of load balancing. A more efficient way is to use a geometric load balancing algorithm.

3.1 Load Balancer

The load balancer uses the recursive coordinate bisection algorithm for load balancing. We support weighted load balancing given the weights for each object as well as the weight for each process. Thus, the processes to be executed on the GPUs can be given a higher weight and those executing on the CPUs can be given a lower weight.

3.1.1 Recursive coordinate bisection

The load balancer uses recursive coordinate bisection (RCB) for load balancing. As the name suggests, this algorithm works by bisecting the domain recursively until the required number of partitions are not created. However, as objects as well as processes may have different weights, the implementation becomes a little more complicated.

Before the load balancer is called for the first time, all the data is owned by the root process. When the load balancer is called, only the root process starts execution, all other process continue to wait for data. The load balancer maintains a list of processes the data needs to be distributed to. This list of process is then bisected making two process lists, left and right. The cumulative weight for the left and right process lists is then calculated. We call the cumulative weight of the left list as the target weight.

Following this, the longest dimension for the list of objects is found and the objects are sorted along that dimension. A prefix scan [Blelloch, 1990] then calculates the index in the sorted list of objects where the cumulative sum of the weights of the objects to the left of that index is less than or equal to the target weight. The object list is then partitioned into left and right

parts. The right part of the list of objects as well as the list of processes is then transfered to the first process in the right list of processes. Once the data is transfered, the process to which it is transfered follows the same process. This continues until the list of processes received by a process has only a single element. When the call is completed, objects are distributed among all processes according to the weights.

Note that the number of data transfers in the load balance call is equal to the number of processes. However the amount of data sent in each of this transfer is much greater than the amount of data that is finally owned by each process. Thus, we want the minimize the amount of data being distributed during the load balance call. Thus, we distribute only the coordinates, the weights and the IDs of the objects during the call as these are the only properties required for load balancing. Once the call is completed, the IDs are then gathered to the master process. Thus, the master process knows the IDs of the objects and the process that ID belongs to. A communication plan is the created using this information and the plan is used for distributing other properties, thus minimizing the amount of data transfers.

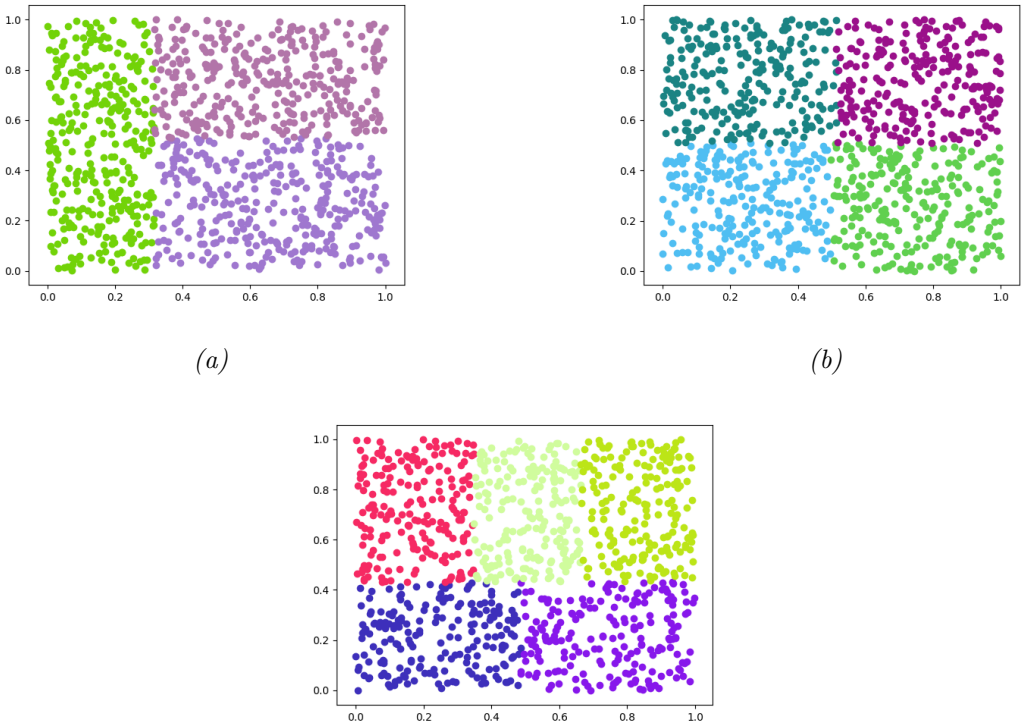
The next time the load balancer is called, the data is gathered by the root process and load balance is called. As we only need the coordinates, weights and IDs for load balancing, only those properties are gathered. When the load balancing is completed, the object IDs are again gathered by the root. Thus, the root knows the old assignment of processes and the new assignment of processes for each object. The new process assignments are then sent to the corresponding processes and are used for creating a communication plan which is then used for the transfer of other properties.

Figure 3.1 shows the result of load balancing on a 2D dataset for different number of partitions. Figure 3.2 shows the result of load balancing on a 2D dataset for different number of partitions with weighted processes.

3.2 Cell based load balancing

The most computationally expensive part of the load balancing algorithm explained above is the sorting of the list of objects along the longest dimension because of the scale of problems on multiple GPUs. Also, the data transfers for coordinates, IDs and weights can also be expensive for a large number of objects. However, because of the large scale of the problems, most of the time a single object will have a small weight. Thus, the objects can be clustered into cells and these cells be instead used as objects for load balancing. The centroids of the cells are used as the coordinates. The weight of a cell is the cumulative sum of the weights of the objects in the cell. Associated to each cell is a cell ID which is used as the ID for load balancing. This reduces the number of objects to be distributed while load balancing as well as the number of objects to be sorted.

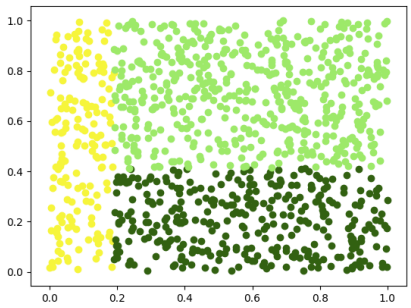
After the cells are distributed, the cell IDs are gathered. From the information of which cell ID belongs to which process, the object IDs contained in those cells are mapped to their corresponding processes and a communication plan is created which is used to transfer the object IDs and coordinates of the objects along with other properties.



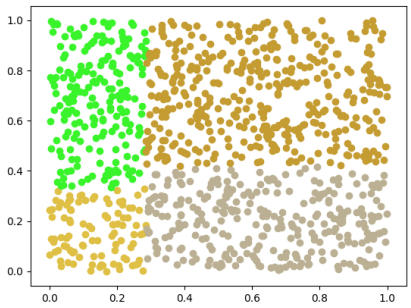
(a) *(b)* *(c)*

Figure 3.1: Result of load balancing on a 2D dataset for (a) 3 (b) 4 and (c) 5 equal partitions.

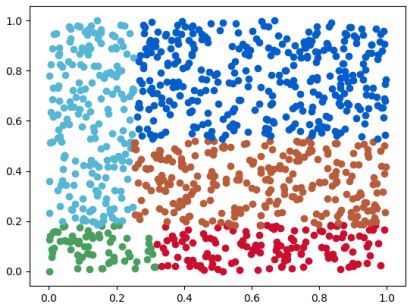
This algorithm for load balancing is currently being used in PySPH. However, it is useful in other applications with a large number of objects as well. We have abstracted it out into our load balancer for the users.



(a)



(b)



(c)

Figure 3.2: Result of load balancing on a 2D dataset for (a) 3 (b) 4 and (c) 5 partitions with weighted processes.

Chapter 4

The framework

Apart from the load balancer which was discussed in the previous chapter, the other components of our framework are discussed in brief in the following sections. Apart from these components, Compyle is a major part of the framework which is discussed in appendix A. All arrays used in our framework as well as arrays created by users should be Compyle arrays. Even though users can write separate functions/kernels for execution on CPUs and GPUs and call them based on the rank of the process, it is recommended that the functions/kernels be also written using Compyle's builtin primitives wherever possible.

It is also worth noting that our framework can be used for running any n-dimensional application without additional overhead. Using the templating feature in Compyle, appropriate kernels are generated at runtime based on the dimensionality of the problem.

4.1 Communication framework

The `Comm` class is the base of all communications in our framework. The `Comm` class takes a list of processes as input and builds a communication plan. The plan is then used to effect the data transfers on any array. This is especially useful in applications where each object has a list of properties associated with it. The plan once created can be used for transferring all the required properties. The plan also calculates the number of objects the current process is going to receive from other processes. This can be used to allocate receiving buffers of appropriate sizes.

`Comm` also supports inverting the plan which can be used to reverse the data transfer operation. This can be useful when the objects are to be first distributed across processes, then some computation is to be performed on the objects by the respective processes and the properties then need to be gathered back to the master process.

We use the `Comm` class in `ObjectExchange` which handles all the data transfers.

4.2 Cell Manager

The `CellManager` class is used for cell based load balancing. The cell manager handles the generation of cells given the list of objects. It also handles the calculation of the domain bounds.

The cells are generated by associating a tuple of ID's to each cell. This tuple is then flattened to get a unique key for each cell. These keys may not be continuous and may have empty cells. To get rid of these empty cell, each non-empty cell is given a cell ID. These cell ID's are continuous and are used as ID's for load balancing.

4.3 Partition Manager

The `PartitionManager` class is the main point of interface for the users. The `update` method of this class is used for load balancing or migrating objects based on the frequency of load balancing specified by the user. The load balance as well as the migrate objects call returns a plan which is passed to `ObjectExchange` which handles the data transfers. Migration of objects uses a Cell Map to find the new process an object has migrated to.

4.4 Cell Map

The `CellMap` class provides a map from the cell ID to the process that cell belongs to. Every process owns a copy of the cell map. The cell map is generated by the master process after load balancing and is broadcasted to other processes.

The cell map is used for migration of particles. When the migrate objects method is called, the new cell ID of all objects is calculated. Using the cell map, the process to which this object now belongs can be found. A communication plan is then created using the new process list and passed to `ObjectExchange`.

4.5 Object Exchange

This class provides the interface for writing data transfer routines to the users. It uses the communication framework underneath to effect these data transfers. Users can create their own class using this as the base class and overload certain methods to write their desired data transfer calls using either the helper methods built into `ObjectExchange` or by using pure MPI calls.

`ObjectExchange` has two methods that can be overloaded, transfer and gather. The gather method, if not overloaded will by default call the transfer method with an inverted plan.

Following is an example of a simple elementwise operation carried out using our framework. The below example can run on multiple GPUs. However, if the backend is set appropriately, it can also run on multiple CPUs or hybrid systems.

```
from compyle.api import annotate, Elementwise, wrap, get_config, declare
from pyzoltan.hetero.partition_manager import PartitionManager
from pyzoltan.hetero.cell_manager import CellManager
from pyzoltan.hetero.object_exchange import ObjectExchange
from pyzoltan.hetero.utils import make_context, only_root
from compyle.cuda import get_context
```

```python
import compyle.array as carr
import numpy as np


@annotate
def f(i, x, y, z):
    z[i] = x[i] * x[i] + y[i] * y[i]


class MyObjectExchange(ObjectExchange):
    def __init__(self, x, y, z, backend):
        self.x = x
        self.y = y
        self.z = z
        self.backend = backend

    def transfer(self):
        x_new = carr.empty(self.plan.nreturn, np.float32,
                           backend=self.backend)
        y_new = carr.empty(self.plan.nreturn, np.float32,
                           backend=self.backend)
        z_new = carr.empty(self.plan.nreturn, np.float32,
                           backend=self.backend)

        self.plan.comm_do_post(self.x, x_new)
        self.plan.comm_do_post(self.y, y_new)
        self.plan.comm_do_post(self.z, z_new)

        self.plan.comm_do_wait()

        self.x = x_new
        self.y = y_new
        self.z = z_new


class Solver(object):
    def __init__(self, n):
        self.backend = 'cuda'
        self.ctx = make_context()
        self.func = Elementwise(f, backend=self.backend)
        self.n = n
        self.x, self.y, self.z = None, None, None
        self.init_arrays()
        self.init_partition_manager()

    def init_partition_manager(self):
```

```
        self.pm = PartitionManager(2, np.float32, backend=self.backend)
        self.oe = MyObjectExchange(self.x, self.y, self.z, self.backend)
        cm = CellManager(2, np.float32, 0.1, num_objs=self.n ** 2,
                         backend=self.backend)
        self.pm.set_object_exchange(self.oe)
        self.pm.set_cell_manager(cm)
        self.pm.setup_load_balancer()

    @only_root
    def init_arrays(self):
        x, y = np.mgrid[0:1:self.n*1j, 0:1:self.n*1j]
        x, y = x.ravel(), y.ravel()
        z = np.zeros_like(x)
        x, y, z = x.astype(np.float32), y.astype(np.float32), z.astype(np.float32)
        self.x, self.y, self.z = wrap(x, y, z, backend=self.backend)

    def calculate(self, x, y, z):
        self.func(x, y, z)

    def solve(self):
        self.pm.update(self.oe.x, self.oe.y, migrate=False)
        self.calculate(self.oe.x, self.oe.y, self.oe.z)
        self.oe.gather()

    @only_root
    def check(self):
        x = self.oe.x.get()
        y = self.oe.y.get()
        z = self.oe.z.get()
        z_calc = x**2 + y**2
        assert np.allclose(z, z_calc)


def run():
    solver = Solver(3000)
    solver.calculate()
    solver.check()
    solver.ctx.pop()


if __name__ == "__main__":
    run()
```

Chapter 5

Analysis

In this chapter we will look at the performance of various components of our framework. We will also look at a simple benchmark problem we use to simulate the computations in a typical SPH simulation and see it's performance with our framework.

5.1 Load balancing

Figure 5.1 shows the time taken for load balancing on the SpaceTime cluster with number of cores going from 1 to 80 over two nodes and 3 different number of objects in 2D. Each node has 40 cores of Intel Skylake 6148 2.4GHz processor. It can be observed that the time taken for load balancing increases steeply for the first few cores and the rate of increase of time decreases as the number of cores increase. This is because as the number of cores increase, more number of processors execute in parallel at each level, thus making the lower levels of the RCB faster.

Figure 5.2 shows the time taken for load balancing for a 2D dataset with 19 million points with the cell size used for out cell based load balancing with 80 cores over two nodes of the SpaceTime cluster. It can be seen that the time taken for load balancing decreases as the cell size increases. However, as the cell size increases, the accuracy of equal distribution of load decreases. For example, with a cell size of 0.1, 1000 cells are to be distributed among 80 processes which will rarely result in a uniform distribution of load.

We will now look at the performance of a simple mandelbrot problem as the number of cores are increased from 1 to 80 across 2 nodes of the SpaceTime cluster and also as the number of cores are increased from 1 to 28 on an Intel i9-9940X 3.3GHz processor. Figure 5.5 shows the plot of time vs number of cores for the benchmark described above.

5.2 Migration of objects

For analysing the performance of our migrate objects routine, we setup a problem with a uniform distribution of particles in 2D rotating about the origin in a circle with speed proportional to their distance from the origin. Each of this particle does a neighbor finding calculation over 3000 particles which is roughly the amount of calculation required for a typical SPH simulation. We run this problem for 40 iterations with the frequency of load balancing being 20 iterations.

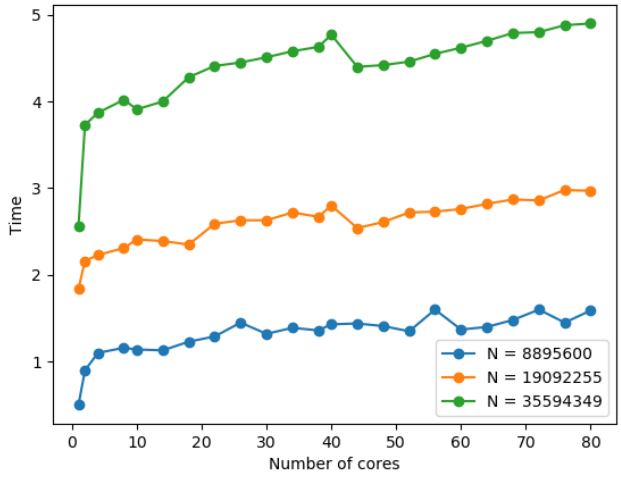


Figure 5.1: Time vs number of nodes for load balancing on SpaceTime

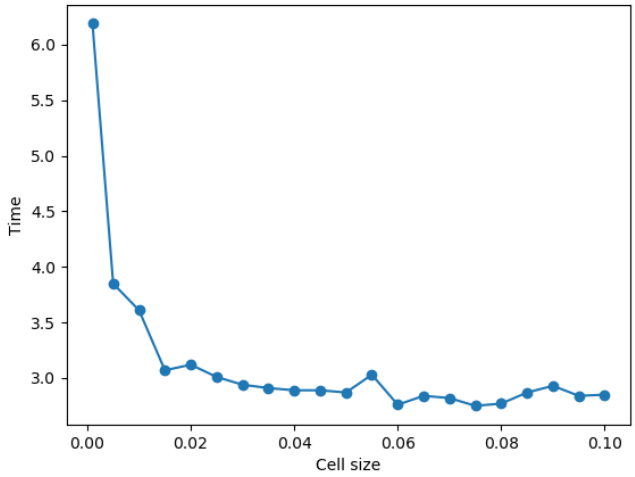
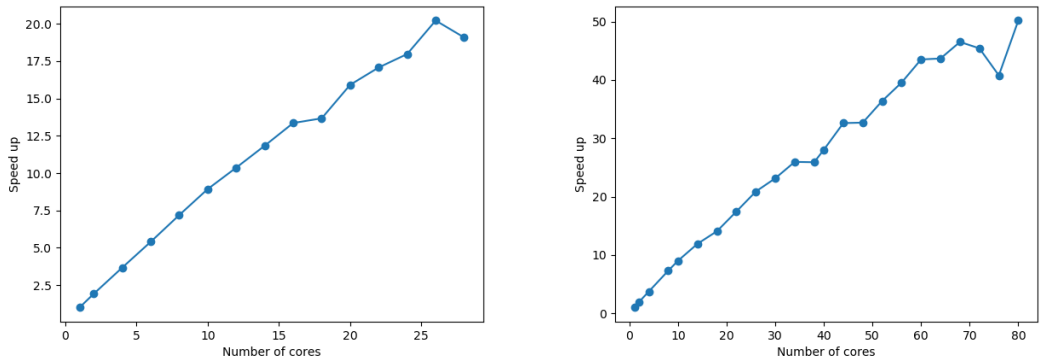


Figure 5.2: Time vs cell size for load balancing on SpaceTime



(a) *(b)*

Figure 5.3: Speed up vs number of cores for mandelbrot problem on (a) Intel i9 processor (b) SpaceTime cluster

The profiling output for our benchmarking problem run on 40 cores of a single node on the SpaceTime cluster showed th following result. The update method of the Partition Manager takes 85.54% of the time whereas the actual calculation takes only 14% of the time. This is mainly because of the inefficient implementation of the migrate objects routine. The new processes to which the objects belong to are found and the process list is sorted for making the communication plan. While the slow sorting of objects during the load balancing was fixed by our cell based approach, the sorting of process list during migration of objects requires the sorting of the entire list of objects belonging to that process. This can be extremely slow for small number of processes as each process owns a large amount of data. Infact this will be the bottleneck for a large number of problems as the sorting itself is $O(N \log N)$. Apart from this, the alignment of every array to be sent during the transfer call is also expensive as the entire array of size equal to the total number of objects in a process has to be aligned.

This problem can however be solved by observing that the number of objects moving from one partition to another after every timestep is much smaller than the total number of objects in that partition. Thus, if only the objects that have moved to the new process are used for building the communication plan and for the transfers, the effective size of the array to be sorted and aligned becomes much smaller. To efficiently align the data however, a swap based approach is required which we are still investigating.

We will now look at the performance of the same problem but without migration of the objects. Figure 5.4 shows the speed up acheived by using 2 GPUs instead of a single GPU as the number of particles are increased from 1 million to 36 million. The scale up on Intel i9 is showed for 1 million particles and on SpaceTime is showed for 4 million particles.

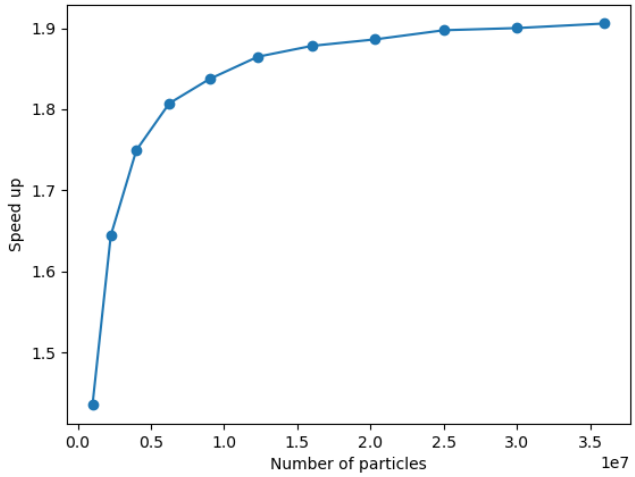
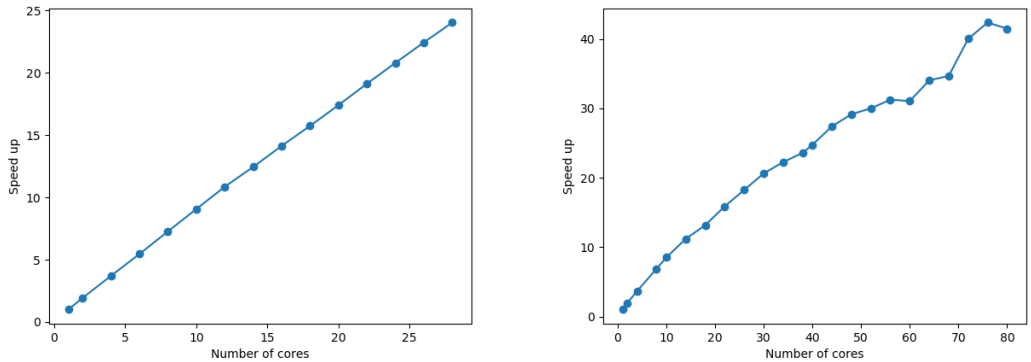


Figure 5.4: Speed up vs number of particles for a 2 GPU setup



(a) *(b)*

Figure 5.5: Speed up vs number of cores for described problem on (a) Intel i9 processor (b) SpaceTime cluster

Chapter 6

Future work

In chapter 2 we looked at the PyZoltan API for dynamic load balancing and communication. We also looked at CUDA aware MPI and it's features. We then looked at the load balancing algorithm we used for our framework and it's results on a 2D dataset. We also looked at the cell based load balancing approach. We then looked at the design of our framework and its different components. We also looked at a simple example for running an elementwise operation using our framework.

In the last chapter we looked at the performance of our framework and came to a conclusion that the migrate objects routine needs to be optimized. We also provided a strategy to potentially optimize this method. We have only analysed the GPU part of our framework on a small 2 GPU setup. The SpaceTime cluster has 8 nodes with an Nvidia P100 on each node. We plan on running the benchmarks on this setup to study the performance of each component on a large scale GPU system.

While a lot of applications can built using the framework in its current state, it is still not complete. In a large number of scientific applications, the knowledge of neighboring point properties are required for updating the properties of the current point. For this to work on the boundaries of a partition, a layer of ghost objects has to be received from the neighboring partitions. We currently do not support this operation. However with the infrastructure we have built, it is fairly straightforward. After load balancing, for each cell in a partition, the keys of all neighboring cells are found. Then using the cell map, the cell IDs corresponding to each of those neighboring cells are found. Then again using the cell map, the process to which these neighboring cells can be found. If that process is not the current process, then the object is a neighbor of that process and has to be sent to that process. Thus for every cell the process to which it is to be sent as a ghost can be found and a communication plan can be created.

We also plan on using our framework for adding heterogeneous support in PySPH.

Appendix A

Compyle

Working on providing support for heterogeneous computing leaves us with a unique problem. Although the algorithm that we use is the same whether it runs on the CPU or the GPU, the implementation differs if we choose to use a CUDA backend. We use a cython backend to implement serial algorithms in PySPH. Cython [Behnel et al., 2011] uses the Python-C API to generate and compile C code. This means that the same algorithm now needs to be written in CUDA and in C/Cython separately for it to work on heterogeneous platforms. This will result in unnecessary code duplication and clutter the source with a lot of similar looking code. Apart from this, PyCUDA and PyOpenCL require you to write and pass the kernels in the form of strings of C code which makes the codebase really messy.

Compyle is a framework that allows you to execute pure python on a variety of HPC platforms. Currently Compyle supports 3 backends; Cython, OpenCL and CUDA. OpenMP support for cython is also supported. Compyle allows users to write code with a restricted python syntax which is then transpiled, compiled and executed on single or multiple core on a CPU or a GPU based on user's choice of backend.

Compyle provides support for custom kernels as well as 3 parallel primitives of map, reduce and scan motivated by PyOpenCL and PyCUDA [Klöckner et al., 2012]. Map or elementwise, as the name suggests is used for doing elementwise computations on the input. Reduction is an operation that maps an input array to a single value using a binary operator. Scan is a generalization of cumulative sum or prefix sum with any binary operator. PyCUDA currently doesn't support a generalized scan. We have implemented a general scan in Compyle based on the PyOpenCL scan implementation.

Here is an example of what Compyle is capable of doing.

```
import numpy as np
from pysph.compyle.api import annotate, Elementwise, get_config

def axpb(i, x, y, a, b):
    y[i] = a[i]*sin(x[i]) + b[i]

# Setup the input data
n = 1000000
```

```
x = np.linspace(0, 1, n)
y = np.zeros_like(x)
a = np.random.random(n)
b = np.random.random(n)

# Use OpenMP
get_config().use_openmp = True

# Now run this in parallel with Cython.
backend = 'cython'
e = Elementwise(axpb, backend=backend)
e(x, y, a, b)
```

This calculates the value of $a \sin x + b$ and stores it in an output array. The `backend` can be changed to 'opencl' or 'cuda' and the same code would run on the GPU with the corresponding backend.

Following is an example of a prefix scan written using Compyle to find the unique elements in a sorted array.

```
import numpy as np
from pysph.compyle.api import annotate, Elementwise, get_config


@annotate
def inp_find_unique(i, arr):
    return 1 if i == 0 or arr[i] != arr[i - 1] else 0


@annotate
def out_find_unique(i, prev_item, arr, out):
    out[prev_item] = arr[i]


def find_unique(arr, out, backend='cython'):
    find_unique_knl = Scan(inp_find_unique, out_find_unique, dtype=np.int32,
                           backend=backend)
    find_unique_knl(arr=arr, out=out)
```

Bibliography

Behnel, S., Bradshaw, R., Citro, C., Dalcin, L., Seljebotn, D., Smith, K., 2011. Cython: The Best of Both Worlds. Computing in Science Engineering 13 (2), 31 –39.

Bhosale, A., 2016. Optimal nearest neighbor searching algorithms for particle based simulations.

Bhosale, A., 2017. Nearest neighbor searching on GPU.

Bird, G. A., 1976. Molecular gas dynamics. NASA STI/Recon Technical Report A 76.

Blelloch, G. E., Nov. 1990. Prefix sums and their applications. Tech. Rep. CMU-CS-90-190, School of Computer Science, Carnegie Mellon University.

Dalcin, L. D., Paz, R. R., Kler, P. A., Cosimo, A., 2011. Parallel distributed computing using python. Advances in Water Resources 34 (9), 1124 – 1139, new Computational Methods and Software Tools.
URL `http://www.sciencedirect.com/science/article/pii/S0309170811000777`

Domínguez, J., Crespo, A., Valdez-Balderas, D., Rogers, B., Gómez-Gesteira, M., 2013. New multi-GPU implementation for smoothed particle hydrodynamics on heterogeneous clusters. Computer Physics Communications 184 (8), 1848 – 1860.
URL `http://www.sciencedirect.com/science/article/pii/S0010465513001057`

Gingold, R. A., Monaghan, J. J., 1977. Smoothed particle hydrodynamics - Theory and application to non-spherical stars. Monthly Notices of the Royal Astronomical Society 181, 375–389.

Goswami, P., Schlegel, P., Solenthaler, B., Pajarola, R., 2010. Interactive SPH Simulation and Rendering on the GPU. In: Proceedings of the 2010 ACM SIGGRAPH/Eurographics Symposium on Computer Animation. SCA '10. Eurographics Association, Aire-la-Ville, Switzerland, Switzerland, pp. 55–64.
URL `http://dl.acm.org/citation.cfm?id=1921427.1921437`

Klöckner, A., Pinto, N., Lee, Y., Catanzaro, B., Ivanov, P., Fasih, A., 2012. PyCUDA and PyOpenCL: A Scripting-Based Approach to GPU Run-Time Code Generation. Parallel Computing 38 (3), 157–174.

Krog, Ø. E., Elster, A. C., 2012. Fast GPU-Based Fluid Simulations Using SPH. In: Jónasson, K. (Ed.), Applied Parallel and Scientific Computing. Springer Berlin Heidelberg, Berlin, Heidelberg, pp. 98–109.

NVIDIA, 2013. An Introduction to CUDA-Aware MPI.

Prabhu Ramachandran, 2016. PySPH: a reproducible and high-performance framework for smoothed particle hydrodynamics. In: Sebastian Benthall, Scott Rostrup (Eds.), Proceedings of the 15th Python in Science Conference. pp. 122 – 129.

Puri, K., Ramachandran, P., Godbole, P., Feb 2013. Load balancing strategies for sph. In: 2013 National Conference on Parallel Computing Technologies (PARCOMPTECH). pp. 1–5.

Savitzky, A., Golay, M. J. E., 1964. Smoothing and Differentiation of Data by Simplified Least Squares Procedures. Analytical Chemistry 36 (8), 1627–1639.
URL https://doi.org/10.1021/ac60214a047

Xia, X., Liang, Q., 2016. A GPU-accelerated smoothed particle hydrodynamics (SPH) model for the shallow water equations. Environmental Modelling & Software 75, 28 – 43.
URL http://www.sciencedirect.com/science/article/pii/S1364815215300608